

INATEL — Instituto Nacional de Telecomunicações

C213 — Sistemas de Embarcados

Projeto Prático C213 — Sistemas Embarcados

Identificação de Processos e Sintonia de Controladores PID

Grupo 5 — Métodos: IMC + ITAE

Eduardo Pereira Valias

Vinicius Telles e Silva

1. Introdução

Este relatório apresenta o desenvolvimento de uma aplicação de Identificação de Sistemas e Controle PID, realizada no contexto da disciplina C213 - Sistemas de Controle Automático. O projeto integra fundamentos matemáticos de controle clássico com uma interface gráfica interativa desenvolvida em Python, utilizando as bibliotecas PyQt5 e python-control.

O sistema permite carregar datasets experimentais de plantas térmicas, identificar o modelo FOPDT (First Order Plus Dead Time), projetar controladores PID por métodos clássicos de sintonia e visualizar o desempenho em malha aberta e fechada.

O Grupo 5 implementou os métodos de sintonia IMC (Internal Model Control) e ITAE (Integral of Time-weighted Absolute Error), conforme definido na Tabela 1 do guia do projeto.

2. Arquitetura da Aplicação

A aplicação segue o padrão arquitetural Modelo-Vista-Controlador (MVC), separando as responsabilidades em três camadas distintas:

2.1 Modelo (Model)

O modelo é implementado no arquivo `PID_logica.py`, que contém a classe `logicaPID` com todos os algoritmos matemáticos do sistema:

- Identificação de sistemas pelos métodos de Smith e Sundaresan;
- Cálculo do Erro Quadrático Médio (EQM);
- Sintonia PID pelos métodos IMC e ITAE;
- Verificação de estabilidade por análise de polos;
- Simulação de resposta em malha aberta e fechada.

2.2 Vista (View)

A interface gráfica é implementada em PyQt5 com múltiplas abas, oferecendo:

- Aba Identificação: carregamento de datasets `.mat` e exibição dos parâmetros identificados (k , τ , θ);
- Aba Controle PID: seleção do método de sintonia, configuração do SetPoint e exibição das métricas de desempenho;
- Visualização dos gráficos com marcadores em pontos de interesse (t_r , t_s , M_p).

2.3 Controlador (Controller)

O controlador gerencia as interações entre a interface e a lógica de negócio, incluindo validação de entradas, disparo dos algoritmos de identificação e sintonia, e atualização dos gráficos em tempo real.

Destaca-se o mecanismo de verificação de estabilidade: antes de aplicar parâmetros manuais, o sistema analisa os polos do sistema em malha fechada. A estabilidade é confirmada apenas quando todos os polos possuem parte real negativa, isto é, estão no semiplano esquerdo do plano complexo:

$$\text{Re}(p_i) < 0, \quad \forall i$$

3. Identificação do Sistema

A identificação de sistemas consiste em determinar um modelo matemático que represente adequadamente a dinâmica da planta a partir de dados experimentais. Neste projeto, o modelo adotado é o FOPDT (First Order Plus Dead Time), descrito pela função de transferência:

$$G(s) = K \cdot e^{(-\theta s)} / (\tau s + 1)$$

onde K é o ganho estático, τ é a constante de tempo e θ é o atraso de transporte (tempo morto).

3.1 Método de Smith

O método de Smith utiliza dois instantes da resposta ao degrau para estimar os parâmetros do modelo:

- t_1 : instante em que a saída atinge 28,3% do valor final;
- t_2 : instante em que a saída atinge 63,2% do valor final.

Com esses pontos, os parâmetros são calculados como:

$$\tau = 1,5 \cdot (t_2 - t_1) \quad \theta = t_2 - \tau$$

3.2 Método de Sundaesan

O método de Sundaesan é uma alternativa mais robusta, utilizando os instantes em que a saída atinge 35,3% e 85,3% do valor final:

$$\tau = (2/3) \cdot (t_2 - t_1) \quad \theta = 1,3 \cdot t_1 - 0,29 \cdot t_2$$

3.3 Erro Quadrático Médio (EQM)

Para determinar qual dos dois métodos representa melhor o processo real, calcula-se o EQM (ou RMSE) entre a resposta do modelo identificado e os dados experimentais:

$$EQM = \sqrt{(\sum (y_{\text{modelo}} - y_{\text{exp}})^2 / N)}$$

O método com menor EQM é automaticamente selecionado como o modelo base para a sintonia do controlador.

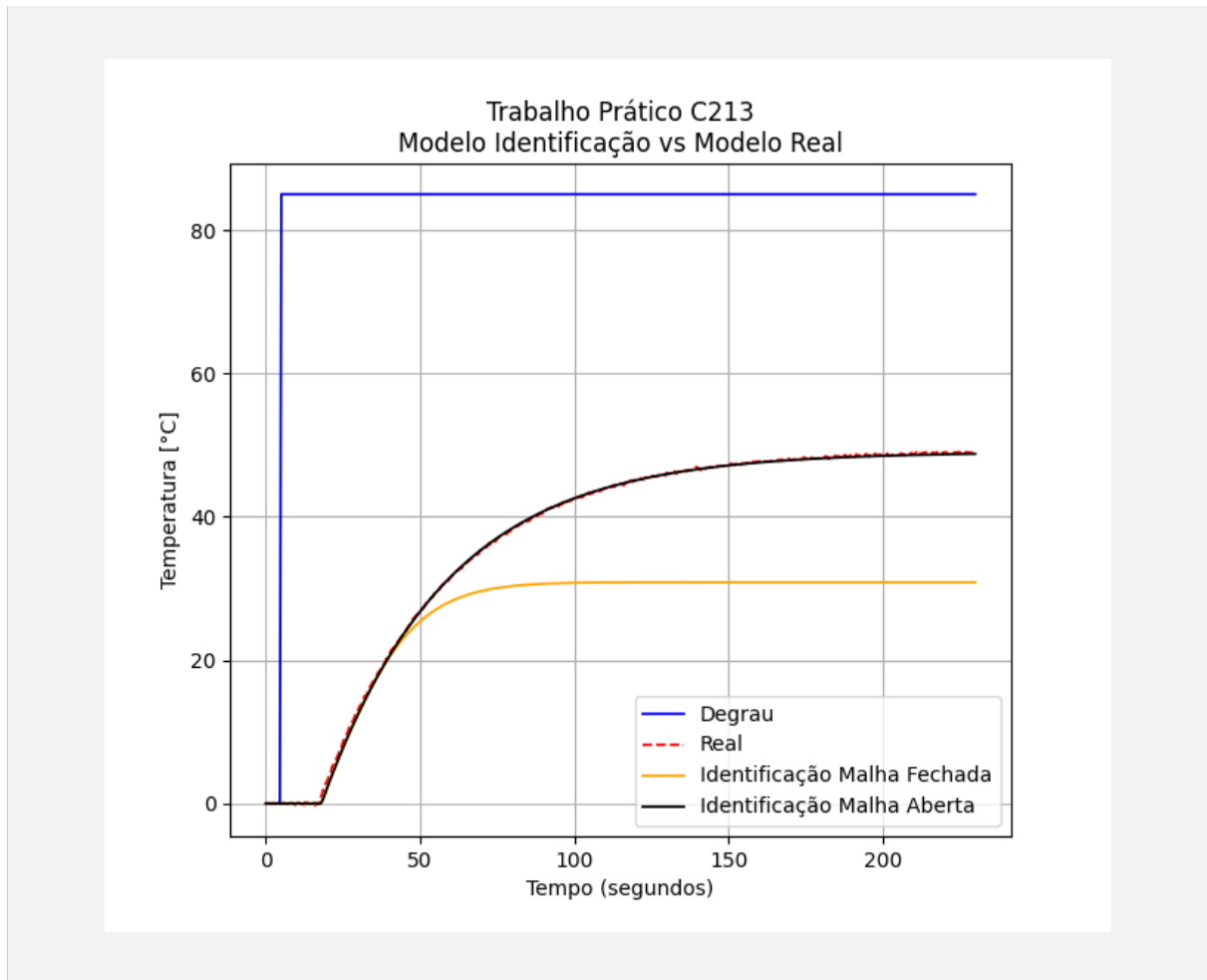


Figura 1 — Resposta ao degrau com identificação pelo método de Smith.

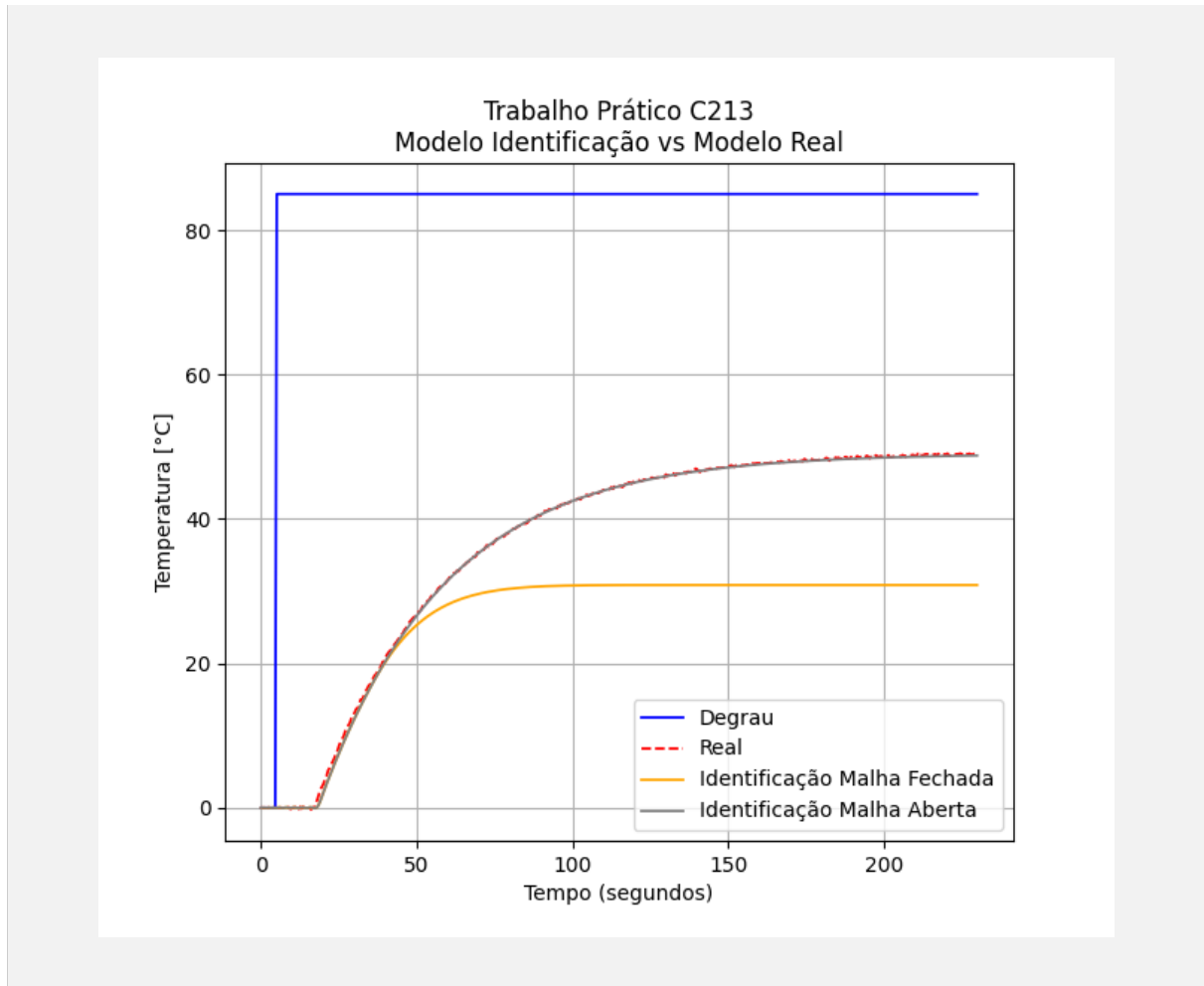


Figura 2 — Resposta ao degrau com identificação pelo método de Sundaresan.

4. Sintonia do Controlador PID

O controlador PID é descrito pela função de transferência paralela:

$$C(s) = K_p \cdot \left(1 + \frac{1}{T_i \cdot s} + T_d \cdot s\right)$$

A sintonia consiste em determinar os parâmetros K_p , T_i e T_d que resultem no desempenho desejado. O Grupo 5 implementou dois métodos: IMC e ITAE.

4.1 Método IMC (Internal Model Control)

O método IMC baseia-se na filosofia de que, se o modelo interno da planta for perfeito, o controlador pode cancelar exatamente a dinâmica do processo. A parametrização IMC para um modelo FOPDT com aproximação de Padé de 1ª ordem para o tempo morto resulta nas seguintes expressões:

$$K_p = (\tau + 0,5\theta) / [K \cdot (\lambda + 0,5\theta)]$$

$$T_i = \tau + 0,5\theta$$

$$T_d = (\tau \cdot \theta) / (2\tau + \theta)$$

4.1.1 Justificativa do parâmetro λ

O parâmetro λ (lambda) é a constante de tempo de malha fechada desejada e representa o principal grau de liberdade do método IMC. Sua escolha define o compromisso fundamental entre velocidade de resposta e robustez do sistema:

- Valores pequenos de λ ($\lambda \rightarrow 0$): resposta mais rápida, porém com maior sensibilidade a ruídos de medição e a incertezas no modelo identificado;
- Valores grandes de λ ($\lambda \gg \theta$): resposta mais lenta e conservadora, porém com maior robustez e menor tendência ao overshoot.

No código implementado, adotou-se como valor padrão $\lambda = \theta$ (igual ao tempo morto do processo), que é uma escolha clássica na literatura de controle por IMC. Essa escolha oferece um equilíbrio satisfatório: a resposta resultante é suficientemente rápida para acompanhar variações no setpoint sem comprometer a robustez frente a perturbações. Para o processo analisado, esse valor deve ser verificado e, se necessário, ajustado para obter o desempenho desejado, o que pode ser feito diretamente na interface pelo campo λ .

4.2 Método ITAE (Integral of Time-weighted Absolute Error)

O critério ITAE minimiza o índice de desempenho $J = \int t \cdot |e(t)| dt$, penalizando erros persistentes em t crescente. As fórmulas empíricas utilizadas são:

$$\begin{aligned} K_p &= (A/K) \cdot (\theta/\tau)^B & [A=0,965; B=-0,85] \\ T_i &= \tau / (C + D \cdot (\theta/\tau)) & [C=0,796; D=-0,147] \\ T_d &= \tau \cdot E \cdot (\theta/\tau)^F & [E=0,308; F=0,929] \end{aligned}$$

Esse método tende a produzir respostas com menor erro acumulado ao longo do tempo, sendo indicado para processos onde a precisão em regime permanente é prioritária.

5. Métodos Implementados no Código

Esta seção detalha os métodos efetivamente presentes nos arquivos Python do Grupo 5. A separação entre lógica e interface facilita a manutenção do projeto: o arquivo `PID_logica.py` concentra os cálculos de identificação, sintonia, simulação e estabilidade, enquanto o arquivo `PID-interface.py` concentra a navegação, os eventos da interface, o carregamento de arquivos e a atualização dos gráficos.

5.1 Métodos do arquivo `PID_logica.py`

O arquivo `PID_logica.py` implementa a classe `logicaPID`. Seus métodos foram definidos como estáticos porque executam cálculos matemáticos a partir dos parâmetros recebidos, sem depender do estado visual da interface.

`smith(tempo, degrau, temperatura)` — aplica o método de identificação de Smith. A função remove o valor inicial da temperatura, calcula o ganho estático K , identifica os instantes em que a resposta atinge 28,3% e 63,2% do valor final e, a partir deles, determina τ e θ . Também retorna as amplitudes inicial e do degrau, usadas nas etapas seguintes.

`sundaresan(tempo, degrau, temperatura)` — aplica o método de identificação de Sundaresan. A lógica é semelhante à de Smith, porém utiliza os pontos de 35,3% e 85,3% do valor final para estimar τ e θ , permitindo comparar duas aproximações para o mesmo dataset.

`eqm(y_modelo, y_experimental)` — calcula o erro quadrático médio entre a resposta simulada e os dados experimentais. Esse valor é usado como critério de comparação para escolher automaticamente o modelo de identificação mais adequado antes da sintonia PID.

`metodo_imc(k, tau, theta, lambda_c=None)` — calcula os parâmetros K_p , T_i e T_d pelo método IMC. Quando o usuário não informa λ , o código assume $\lambda = \theta$, adotando uma sintonia equilibrada entre velocidade de resposta e robustez.

`metodo_itae(k, tau, theta)` — calcula K_p , T_i e T_d pelo critério ITAE, utilizando constantes empíricas definidas no próprio código. Esse método busca reduzir o erro persistente ao longo do tempo, penalizando erros que permanecem por mais tempo na resposta.

`calcular_metricas(Kp, Ti, Td, sp, sistemaControle)` — monta o controlador PID, conecta-o em série com a planta identificada, fecha a malha e calcula métricas de desempenho por meio da biblioteca `python-control`. O método retorna tempo de acomodação, tempo de subida, overshoot e erro em regime permanente.

`criar_modelo_sistema_controle(tempo, degrau, temperatura, metodo)` — seleciona Smith ou Sundaresan, calcula K , τ e θ , cria a função de transferência FOPDT da planta, aproxima o atraso por Padé de ordem 20 e gera as respostas simuladas em malha aberta e malha fechada.

`criar_sistema_controle_pid(Kp, Ti, Td, sys_atraso, sp)` — cria a função de transferência do controlador PID, conecta o controlador à planta, fecha a malha e calcula a resposta ao degrau multiplicada pelo setpoint definido.

`verificar_estabilidade(Kp, Ti, Td, sys_atraso)` — verifica a estabilidade da malha fechada antes de aceitar parâmetros manuais. O sistema é considerado estável somente se todos os polos tiverem parte real negativa. Caso T_i seja menor ou igual a zero, ou ocorra algum erro no cálculo, a função retorna falso.

5.2 Métodos do arquivo `PID-interface.py`

O arquivo `PID-interface.py` organiza a aplicação em páginas PyQt5. Cada classe representa uma tela ou o controlador principal da aplicação, conectando botões, campos de entrada, gráficos e métodos da classe `logicaPID`.

5.2.1 Classe `PaginaLogin`

validar_credencial(email, senha) — valida o acesso ao sistema, aceitando apenas o e-mail admin@admin.com e a senha admin.

sizeHint() — define o tamanho recomendado da janela de login, mantendo essa tela menor que as demais telas do sistema.

5.2.2 Classe PaginalIdentSists

sizeHint() — define o tamanho recomendado da página de identificação de sistemas.

open_mat_file() — abre a caixa de seleção de arquivo e permite ao usuário escolher um dataset MATLAB com extensão .mat. Caso um arquivo seja selecionado, seu caminho é exibido na interface e o processamento é iniciado.

load_and_plot_mat(caminho) — carrega o arquivo .mat com scipy.io.loadmat, verifica se existem as variáveis dados_entrada e dados_saida, extrai tempo, degrau e temperatura, calcula os modelos por Smith e Sundaresan, calcula o EQM de cada um, atualiza os campos da interface e libera os botões de exportação e de controle PID.

grafico_comparacao_identificacao_real(tempo, degrau, temperatura, tempoSimulado, temperaturaSimulada, tempo_fechada, amplitude_fechada) — limpa o gráfico atual e plota o degrau de entrada, a resposta real, a identificação em malha aberta e a identificação em malha fechada. Também configura título, eixos, grade e legenda.

exportar_grafico() — abre uma janela para salvar o gráfico de identificação em formato PNG no diretório escolhido pelo usuário.

populate_combo_box() — preenche o ComboBox da interface com os métodos de identificação disponíveis: Smith e Sundaresan.

on_option_selected(index) — atualiza o método de identificação selecionado, recupera os parâmetros já calculados para Smith ou Sundaresan, atualiza K, τ , θ e EQM na tela e redesenha o gráfico correspondente.

5.2.3 Classe PaginaControlePID

toggle_manual_mode(manual) — alterna entre sintonia automática e manual. No modo manual, libera os campos Kp, Ti e Td e desativa a escolha automática de método; no modo automático, bloqueia os campos PID e libera a escolha entre IMC e ITAE.

sizeHint() — define o tamanho recomendado da página de controle PID.

exportar_grafico() — permite salvar o gráfico de resposta do controlador PID em PNG e exibe uma mensagem de confirmação após a exportação.

set_processing_mode(mode) — registra o modo de processamento selecionado e chama a rotina de bloqueio ou liberação dos campos de PID conforme o modo escolhido.

toggle_pid_inputs(enabled) — ativa ou desativa campos de entrada de parâmetros PID, alterando também o estilo visual dos campos.

populate_combo_box() — preenche o ComboBox da página de controle com os métodos de sintonia disponíveis: IMC e ITAE.

on_option_selected(index) — atualiza o método de sintonia selecionado. Quando IMC está ativo no modo automático, o campo λ fica editável; para ITAE ou modo manual, esse campo é bloqueado. Em seguida, os parâmetros são recalculados.

definir_parametros_sistema(k, tau, theta, tempo, temperatura, metodo, sistemaControle, amplitude_degrau) — recebe da tela de identificação os parâmetros do modelo escolhido, armazena os dados necessários para o controle PID, inicializa λ com o valor de θ e define o setpoint inicial a partir da amplitude média do degrau.

calcular_parametros() — lê o setpoint, calcula Kp, Ti e Td pelo método automático escolhido ou pelos valores manuais digitados. No modo manual, chama verificar_estabilidade antes de simular a resposta, impedindo a atualização do gráfico quando os polos indicam instabilidade.

sintonizar(Kp, Ti, Td, t, y, ts, tr, mp, erro) — atualiza os campos da interface com os parâmetros PID calculados e com as métricas de desempenho, além de chamar o método responsável por redesenhar o gráfico da resposta controlada.

grafico_sintonizacao(t, y, sp, ts, tr, mp) — plota a resposta do sistema controlado, o setpoint, a banda de acomodação de $\pm 2\%$, o pico de overshoot e marcações de tempo. Para IMC, destaca o tempo de subida; para ITAE, destaca o tempo de acomodação; no modo manual, apresenta as marcações disponíveis.

5.2.4 Classe PaginaGraphs

sizeHint() — define o tamanho recomendado da tela de gráficos, mantendo consistência visual com as demais telas principais.

5.2.5 Classe AplicacaoPrincipal

mostrar_identificacao() — altera o índice do QStackedWidget para exibir a página de identificação de sistemas.

mostrar_pid() — seleciona automaticamente o modelo com menor EQM entre Smith e Sundaresan, envia os parâmetros para a página de controle PID, calcula a sintonia inicial e navega para a tela de controle.

mostrar_graficos() — altera o índice da aplicação para exibir a página de gráficos.

reiniciar_aplicacao() — limpa os campos de login e retorna a aplicação para a tela inicial.

voltar_pagina() — controla a navegação de retorno. A partir da tela de identificação, reinicia a aplicação; a partir das demais telas, volta para a página anterior.

ajustar_tamanho_janela() — redimensiona a janela principal de acordo com o sizeHint da página atualmente exibida.

login_invalido() — exibe uma caixa de diálogo informando que as credenciais inseridas são inválidas.

try_login() — lê e-mail e senha digitados pelo usuário, valida as credenciais e direciona para a identificação de sistemas quando o login é aceito; caso contrário, chama login_invalido.

5.3 Fluxo técnico entre os métodos

O fluxo técnico começa em open_mat_file(), que chama load_and_plot_mat() após a seleção do dataset. Em seguida, load_and_plot_mat() utiliza criar_modelo_sistema_controle() para gerar os modelos de Smith e Sundaresan e eqm() para comparar os resultados. Quando o usuário avança para o controle PID, mostrar_pid() escolhe o menor EQM e envia os dados para definir_parametros_sistema(). Por fim, calcular_parametros() usa metodo_imc() ou metodo_itae(), chama criar_sistema_controle_pid(), calcula as métricas com calcular_metricas() e atualiza o gráfico por meio de sintonizar() e grafico_sintonizacao().

No modo manual, o mesmo fluxo é preservado, mas antes da simulação o método verificar_estabilidade() é chamado. Essa etapa evita que valores digitados pelo usuário gerem uma resposta instável sem aviso, reforçando a segurança da interface durante a apresentação e os testes.

6. Resultados e Análise Comparativa

6.1 Parâmetros Identificados

Após o carregamento do dataset e execução dos métodos de identificação, os parâmetros FOPDT obtidos foram:

Método	K	τ (s)	θ (s)
--------	---	------------	--------------

Smith	0.5898	40.5	18.0
Sundaresan	0.5898	40.6667	18.165

Tabela 1 — Parâmetros FOPDT identificados pelos dois métodos.

6.2 Desempenho do Sistema Controlado

A tabela abaixo compara as métricas de desempenho obtidas com os dois métodos de sintonia implementados para o Grupo 5, considerando o SetPoint definido:

Método	tr (s)	ts (s)	Mp (%)
IMC	33.4481	78.5603	0
ITAE	33.7334	118.2950	0

Tabela 2 — Métricas de desempenho: IMC vs ITAE.

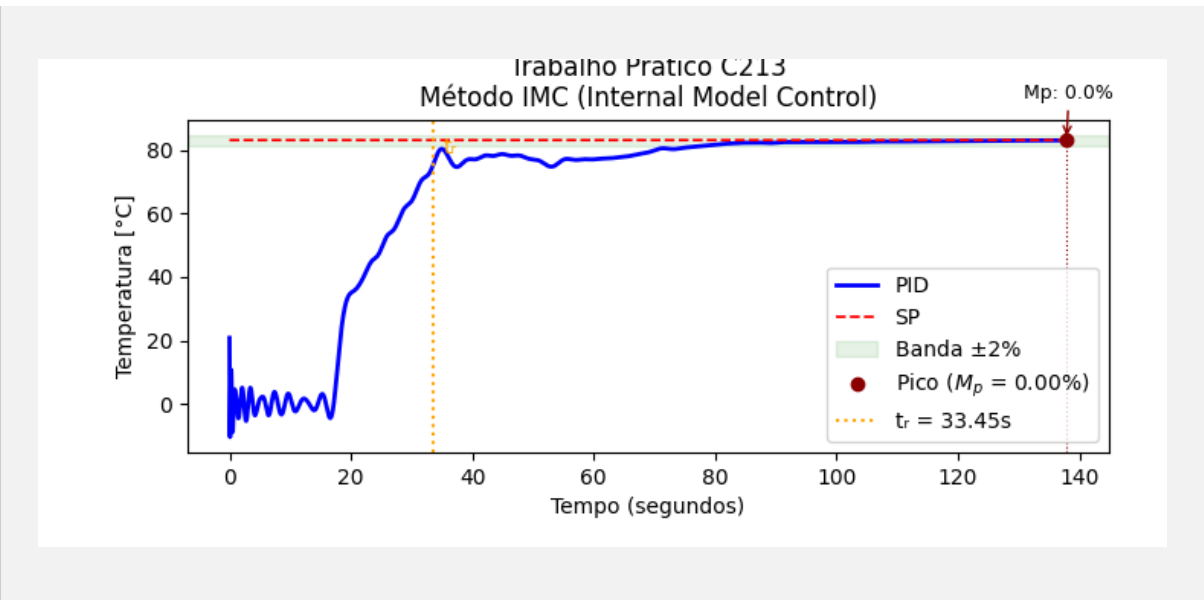


Figura 3 — Resposta ao degrau com controle PID por IMC.

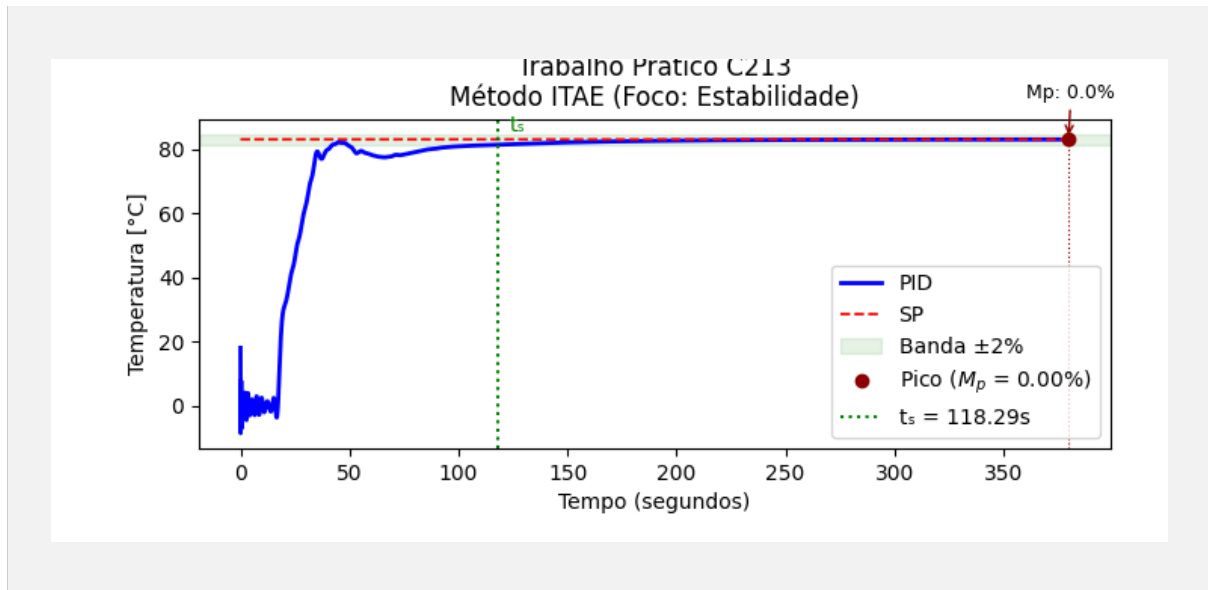


Figura 4 — Resposta ao degrau com controle PID por ITAE.

6.3 Análise Comparativa

A análise comparativa dos dados apresentados na Tabela 2 revela que ambos os controladores atingiram o objetivo de estabilizar a planta térmica com sucesso, eliminando o erro em regime permanente e suprimindo totalmente o sobressinal ($M_p = 0\%$). O tempo de subida (t_r) comportou-se de maneira extremamente similar em ambas as abordagens, com o IMC registrando 33,4481s contra 33,7334s do método ITAE.

A principal distinção de desempenho ocorreu na análise do tempo de acomodação (t_s). O método IMC estabilizou o processo dentro da banda de tolerância em apenas 78,5603s. Em contrapartida, o método ITAE mostrou-se mais conservador e lento na sua atuação integral, exigindo 118,2950s para acomodar o sistema.

Conclui-se que, para este modelo FOPDT em específico, a sintonia por *Internal Model Control* (adotando o parâmetro de robustez $\lambda = \theta$) apresentou um desempenho global superior. O controlador cancelou adequadamente a dinâmica dominante do processo, resultando em uma resposta que alia agilidade de acomodação e robustez.

7. Conclusão

O desenvolvimento deste projeto permitiu uma consolidação prática e visual dos conceitos teóricos de Sistemas de Controle Automático. A construção da aplicação exigiu um planejamento cuidadoso da arquitetura MVC, onde a integração fluida das bibliotecas de cálculo numérico com os componentes visuais da interface representou um dos principais desafios técnicos superados pela equipe. A implementação de rotinas de validação, como a verificação de estabilidade através da análise da parte real dos polos em malha fechada antes da liberação do modo de sintonia manual, reforçou a importância de se projetar softwares de supervisão seguros e tolerantes a erros operacionais.

A análise prática da planta térmica evidenciou como a escolha do algoritmo de sintonia impacta diretamente a dinâmica do chão de fábrica. Embora o método ITAE seja amplamente reconhecido por minimizar erros persistentes no tempo, o método IMC demonstrou-se mais eficiente e veloz para o *dataset* ensaiado, confirmando a eficácia técnica de utilizar o tempo morto como baliza heurística para o parâmetro λ .

Como propostas de melhorias futuras para a expansão deste simulador, vislumbra-se a implementação de algoritmos de identificação para sistemas subamortecidos de segunda ordem, a inclusão de malhas com compensação *anti-windup* para o modo integral e a ampliação dos módulos de importação para suportar extensões de dados mais universais, como arquivos em formato tabular.

Referências

- [1] Smith, C. L., Digital Computer Process Control, Intext Educational Publishers, 1972.
- [2] Sundaresan, K. R., Krishnaswamy, P. R., Estimation of time delay parameters in linear systems, Industrial & Engineering Chemistry Process Design and Development, 1978.
- [3] Rivera, D. E., Morari, M., Skogestad, S., Internal Model Control: PID Controller Design, Industrial & Engineering Chemistry Process Design and Development, 1986.
- [4] Ogata, K., Engenharia de Controle Moderno. 5ª Edição, Pearson, 2010.
- [5] Documentação python-control: <https://python-control.readthedocs.io/>
- [6] Documentação PyQt5: <https://doc.qt.io/qtforpython/>

